

Zcash Protocol Specification

Version v2026.7.0-191-g0fae78 [NU6.2]

Daira-Emma Hopwood[†]
Sean Bowe[‡] — Taylor Hornby[§] — Nathan Wilcox[◇]

September 15, 2026



Abstract. **Zcash** is an implementation of the *Decentralized Anonymous Payment scheme Zerocash*, with security fixes and improvements to performance and functionality. It bridges the existing transparent payment scheme used by **Bitcoin** with a *shielded* payment scheme secured by *zero-knowledge succinct non-interactive arguments of knowledge (zk-SNARKs)*. It attempted to address the problem of mining centralization by use of the *Equihash* memory-hard *proof-of-work* algorithm.

This specification defines the **Zcash** consensus protocol at launch, and after each of the upgrades codenamed **Overwinter**, **Sapling**, **Blossom**, **Heartwood**, **Canopy**, **NU5**, **NU6**, **NU6.1**, and **NU6.2**. It is a work in progress. Protocol differences from **Zerocash** and **Bitcoin** are also explained.

Keywords: anonymity, applications, cryptographic protocols, electronic commerce and payment, financial privacy, proof of work, zero knowledge.

Contents	1
1 Introduction	8
1.1 Caution	9

[†] Zcash Open Development Lab

[‡] Project Tachyon

[§] Independent consultant

[◇] Shielded Labs

¹ Jubjub bird image credit: Peter Newell 1902; Daira-Emma Hopwood 2018.

3 Concepts

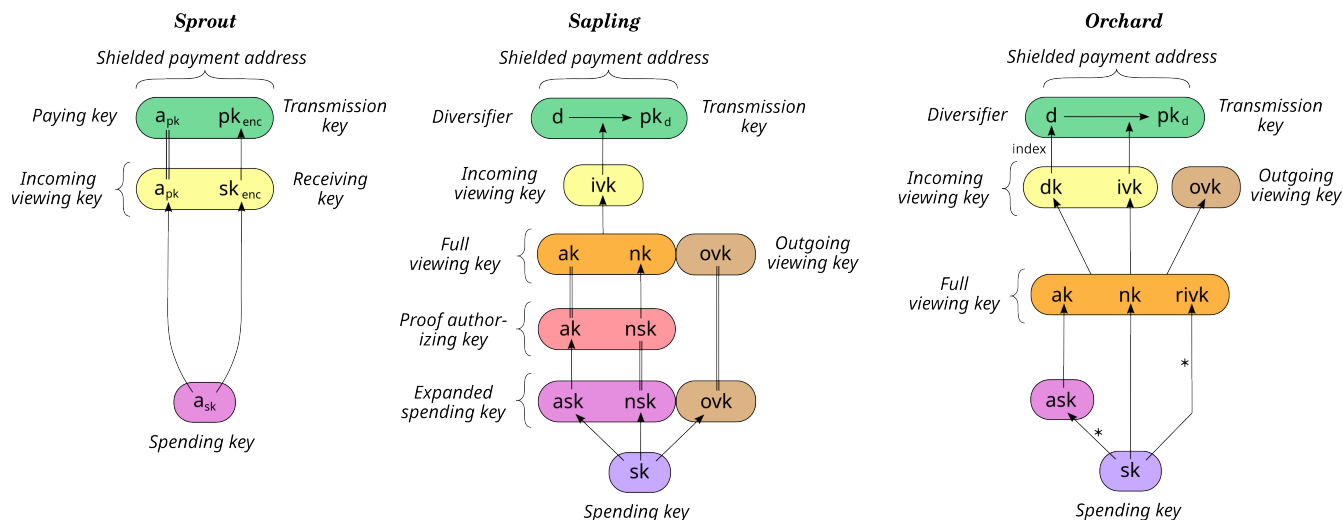
#concepts

3.1 Payment Addresses and Keys

#addressesandkeys

Users who wish to receive shielded payments in the **Zcash** protocol must have a *shielded payment address*, which is generated from a *spending key*.

The following diagrams depict the relations between key components in **Sprout** and **Sapling** and **Orchard**. Arrows point from a component to any other component(s) that can be derived from it. Double lines indicate that the same component is used in multiple abstractions.



[Sprout] A Sprout receiving key sk_{enc} , incoming viewing key $ivk = (a_{pk}, sk_{enc})$, and shielded payment address $addr_{pk} = (a_{pk}, pk_{enc})$ are derived from the spending key a_{sk} , as described in §4.2.1 ‘*Sprout Key Components*’ on p. 38.

[Sapling onward] A Sapling expanded spending key is composed of a Spend authorizing key ask , a nullifier private key nsk , and an outgoing viewing key ovk . From these components we can derive a proof authorizing key (ak, nsk) , a full viewing key (ak, nk, ovk) , an incoming viewing key ivk , and a set of diversified payment addresses $addr_d = (d, pk_d)$, as described in §4.2.2 ‘*Sapling Key Components*’ on p. 38.

The consensus protocol does not depend on how an expanded spending key is constructed. Two methods of doing so are defined:

1. Generate a spending key sk at random and derive the expanded spending key (ask, nsk, ovk) from it, as shown in the diagram above and described in §4.2.2 ‘*Sapling Key Components*’ on p. 38.
2. Obtain an extended spending key as specified in [ZIP-32]; this includes a superset of the components of an expanded spending key. This method is used in the context of a *Hierarchical Deterministic Wallet*.

[NU5 onward] An Orchard spending key sk is used to derive a Spend authorizing key ask , and a full viewing key $(ak, nk, rinvk)$. From the full viewing key we can also derive an incoming viewing key (composed of a diversifier key dk and a $KA^{Orchard}$ private key ivk), an outgoing viewing key ovk , and a set of diversified payment addresses $addr_d = (d, pk_d)$, as described in §4.2.3 ‘*Orchard Key Components*’ on p. 40.

* The derivations of ask and $rinvk$ shown are not the only possibility. For further detail see §4.2.3 on p. 40.

Non-normative note: Most Zcash wallets derive Sapling and Orchard keys and addresses according to [ZIP-32].

The composition of *shielded payment addresses*, *incoming viewing keys*, *full viewing keys*, and *spending keys* is a cryptographic protocol detail that should not normally be exposed to users. However, user-visible operations should be provided to obtain a *shielded payment address*, *incoming viewing key*, or *full viewing key* from a *spending key* or *extended spending key*.

Users can accept payment from multiple parties with a single *shielded payment address* and the fact that these payments are destined to the same payee is not revealed on the *block chain*, even to the paying parties. *However* if two parties collude to compare a *shielded payment address* they can trivially determine they are the same. In the case that a payee wishes to prevent this they should create a distinct *shielded payment address* for each payer.

[Sapling onward] Sapling and Orchard provide a mechanism to allow the efficient creation of *diversified payment addresses* with the same *spending authority*. A group of such addresses shares the same *full viewing key* and *incoming viewing key*, and so creating as many unlinkable addresses as needed does not increase the cost of scanning the *block chain* for relevant *transactions*.

Note: It is conventional in cryptography to call the key used to encrypt a message in an asymmetric encryption scheme a “*public key*”. However, the *public key* used as the *transmission key* component of an address (pk_{enc} or pk_d) need not be publically distributed; it has the same distribution as the *shielded payment address* itself. As mentioned above, limiting the distribution of the *shielded payment address* is important for some use cases. This also helps to reduce reliance of the overall protocol on the security of the cryptosystem used for note encryption (see § 4.19 ‘*In-band secret distribution (Sprout)*’ on p. 68 and § 4.20 ‘*In-band secret distribution (Sapling and Orchard)*’ on p. 70), since an adversary would have to know pk_{enc} or some pk_d in order to exploit a hypothetical weakness in that cryptosystem.

3.2 Shielded Pools and Notes

#notes

A *note* is a representation of value held in a *shielded pool*.

Each *shielded pool* operates according to the rules of a *shielded protocol*, and has its own *treestate* (see § 3.4 ‘*Transactions and Treestates*’ on p. 19). The currently deployed *shielded pools* are the *Sprout pool* and *Sapling pool* and *Orchard pool*.

Each *note* (denoted n) belongs to one of these *shielded pools*. It represents that a value v is spendable by the recipient who holds the *spending key* corresponding to a given *shielded payment address*. The form of a *shielded payment address* is defined by a *shielded protocol*; it can be used to send funds represented by new *notes* to the address, in the *shielded pool(s)* supported by that protocol.

Let MAX_MONEY , $\ell_{\text{PRF}}^{\text{Sprout}}$, $\ell_{\text{PRF}}^{\text{Sapling}}$, ℓ_d , and ℓ_{value} be as defined in § 5.3 ‘*Constants*’ on p. 77.

Let $\text{NoteCommit}^{\text{Sprout}}$ be as defined in § 5.4.8.1 ‘*Note Commitments (Sprout)*’ on p. 98.

Let $\text{NoteCommit}^{\text{Sapling}}$ be as defined in § 5.4.8.2 ‘*Windowed Pedersen commitments*’ on p. 99.

Let $\text{KA}^{\text{Sapling}}$ be as defined in § 5.4.5.3 ‘*Sapling Key Agreement*’ on p. 92.

Let $\text{DiversifyHash}^{\text{Sapling}}$ be as defined in § 5.4.1.6 ‘*DiversifyHash^{Sapling} and DiversifyHash^{Orchard} Hash Functions*’ on p. 81.

Let $\text{NoteCommit}^{\text{Orchard}}$ be as defined in § 5.4.8.4 ‘*Sinsemilla commitments*’ on p. 101.

Let $\text{KA}^{\text{Orchard}}$ be as defined in § 5.4.5.5 ‘*Orchard Key Agreement*’ on p. 93.

Let $\text{DiversifyHash}^{\text{Orchard}}$ be as defined in § 5.4.1.6 ‘*DiversifyHash^{Sapling} and DiversifyHash^{Orchard} Hash Functions*’ on p. 81.

Let $\text{PRF}^{\text{nfOrchard}}$ be as defined in § 5.4.2 ‘*Pseudo Random Functions*’ on p. 89.

Let $q_{\mathbb{P}}$ be as defined in § 5.4.9.6 ‘*Pallas and Vesta*’ on p. 108.

4.2 Key Components

#keycomponents

4.2.1 Sprout Key Components

#sproutkeycomponents

Let $\ell_{a_{sk}}$ be as defined in § 5.3 ‘*Constants*’ on p. 77.

Let PRF^{addr} be a *Pseudo Random Function*, instantiated in § 5.4.2 ‘*Pseudo Random Functions*’ on p. 89.

Let $\text{KA}^{\text{Sprout}}$ be a *key agreement scheme*, instantiated in § 5.4.5.1 ‘*Sprout Key Agreement*’ on p. 91.

A new *Sprout spending key* a_{sk} is generated by choosing a *bit sequence* uniformly at random from $\mathbb{B}^{\ell_{a_{sk}}}$.

a_{pk} , sk_{enc} and pk_{enc} are derived from a_{sk} as follows:

$$\begin{aligned} a_{pk} &:= \text{PRF}_{a_{sk}}^{\text{addr}}(0) \\ sk_{\text{enc}} &:= \text{KA}^{\text{Sprout}}.\text{FormatPrivate}(\text{PRF}_{a_{sk}}^{\text{addr}}(1)) \\ pk_{\text{enc}} &:= \text{KA}^{\text{Sprout}}.\text{DerivePublic}(sk_{\text{enc}}, \text{KA}^{\text{Sprout}}.\text{Base}). \end{aligned}$$

4.2.2 Sapling Key Components

#saplingkeycomponents

Let $\ell_{\text{PRFexpand}}$, ℓ_{sk} , $\ell_{\text{ivk}}^{\text{Sapling}}$, ℓ_{ovk} , and ℓ_d be as defined in § 5.3 ‘*Constants*’ on p. 77.

Let $\mathbb{J}^{(r)}$, $\mathbb{J}^{(r)*}$, $\mathbb{J}_{\star}^{(r)}$, $\text{repr}_{\mathbb{J}}$, and $r_{\mathbb{J}}$ be as defined in § 5.4.9.3 ‘*Jubjub*’ on p. 105, and let $\text{FindGroupHash}^{\mathbb{J}^{(r)*}}$ be as defined in § 5.4.9.5 ‘*Group Hash into Jubjub*’ on p. 107.

Let $\text{PRF}^{\text{expand}}$ and $\text{PRF}^{\text{ockSapling}}$, instantiated in § 5.4.2 ‘*Pseudo Random Functions*’ on p. 89, be *Pseudo Random Functions*.

Let $\text{KA}^{\text{Sapling}}$, instantiated in § 5.4.5.3 ‘*Sapling Key Agreement*’ on p. 92, be a *key agreement scheme*.

Let CRH^{ivk} , instantiated in § 5.4.1.5 ‘*CRH^{ivk} Hash Function*’ on p. 80, be a *hash function*.

Let $\text{DiversifyHash}^{\text{Sapling}}$, instantiated in § 5.4.1.6 ‘*DiversifyHash^{Sapling} and DiversifyHash^{Orchard} Hash Functions*’ on p. 81, be a *hash function*.

Let $\text{SpendAuthSig}^{\text{Sapling}}$, instantiated in § 5.4.7.1 ‘*Spend Authorization Signature (Sapling and Orchard)*’ on p. 98, be a *signature scheme with re-randomizable keys*.

Let $\text{LEBS2OSP} : (\ell : \mathbb{N}) \times \mathbb{B}^{\ell} \rightarrow \mathbb{B}^{\lceil \ell/8 \rceil}$ and $\text{LEOS2IP} : (\ell : \mathbb{N} \mid \ell \bmod 8 = 0) \times \mathbb{B}^{\ell/8} \rightarrow \{0 \dots 2^{\ell} - 1\}$ be as defined in § 5.1 ‘*Integers, Bit Sequences, and Endianness*’ on p. 76.

Define $\mathcal{H}^{\text{Sapling}} := \text{FindGroupHash}^{\mathbb{J}^{(r)*}}(\text{"Zcash_H_"}, \text{""})$

Define $\text{ToScalar}^{\text{Sapling}}(x : \mathbb{B}^{\lceil \ell_{\text{PRFexpand}}/8 \rceil}) := \text{LEOS2IP}_{\ell_{\text{PRFexpand}}}(x) \pmod{r_{\mathbb{J}}}$.

A new *Sapling spending key* sk is generated by choosing a *bit sequence* uniformly at random from $\mathbb{B}^{\ell_{sk}}$.

From this *spending key*, the *Spend authorizing key* $ask : \mathbb{F}_{r_{\mathbb{J}}}^*$, the *proof authorizing key* $nsk : \mathbb{F}_{r_{\mathbb{J}}}$, and the *outgoing viewing key* $ovk : \mathbb{B}^{\lceil \ell_{\text{ovk}}/8 \rceil}$ are derived as follows:

$$\begin{aligned} ask &:= \text{ToScalar}^{\text{Sapling}}(\text{PRF}_{sk}^{\text{expand}}([0])) \\ nsk &:= \text{ToScalar}^{\text{Sapling}}(\text{PRF}_{sk}^{\text{expand}}([1])) \\ ovk &:= \text{truncate}_{\lceil \ell_{\text{ovk}}/8 \rceil}(\text{PRF}_{sk}^{\text{expand}}([2])) \end{aligned}$$

If $ask = 0$, discard this key and repeat with a new sk .

$ak : \mathbb{J}^{(r)*}$, $nk : \mathbb{J}^{(r)}$, and the *incoming viewing key* $ivk : \{1 \dots 2^{\ell_{\text{ivk}}^{\text{Sapling}}} - 1\}$ are then derived as:

$$\begin{aligned} ak &:= \text{SpendAuthSig}^{\text{Sapling}}.\text{DerivePublic}(ask) \\ nk &:= [nsk] \mathcal{H}^{\text{Sapling}} \\ ivk &:= \text{CRH}^{\text{ivk}}(\text{repr}_{\mathbb{J}}(ak), \text{repr}_{\mathbb{J}}(nk)). \end{aligned}$$

If $ivk = 0$, discard this key and repeat with a new sk .

As explained in § 3.1 ‘*Payment Addresses and Keys*’ on p. 14, Sapling allows the efficient creation of many *diversified payment addresses* with the same *spending authority*. A group of such addresses shares the same *full viewing key* and *incoming viewing key*.

To create a new *diversified payment address* given an *incoming viewing key* ivk , repeatedly pick a *diversifier* d uniformly at random from $\mathbb{B}^{[\ell_d]}$ until the *diversified base* $g_d = \text{DiversifyHash}^{\text{Sapling}}(d)$ is not $\perp$. Then calculate the *diversified transmission key* pk_d :

$$pk_d := KA^{\text{Sapling}}.\text{DerivePublic}(ivk, g_d).$$

The resulting *diversified payment address* is $(d : \mathbb{B}^{[\ell_d]}, pk_d : KA^{\text{Sapling}}.\text{PublicPrimeOrder})$.

For each *spending key*, there is also a *default diversified payment address* with a “random-looking” *diversifier*. This allows an implementation that does not expose diversified addresses as a user-visible feature, to use a default address that cannot be distinguished (without knowledge of the *spending key*) from one with a random *diversifier* as above. Note however that the *zcashd* wallet picks *diversifiers* as in [ZIP-32], rather than using this procedure.

Let $\text{first} : (\mathbb{B}^{\mathbb{Y}} \rightarrow T \cup \{\perp\}) \rightarrow T \cup \{\perp\}$ be as defined in § 5.4.9.5 ‘*Group Hash into Jubjub*’ on p. 107. Define:

$$\text{CheckDiversifier}(d : \mathbb{B}^{[\ell_d]}) := \begin{cases} \perp, & \text{if } \text{DiversifyHash}^{\text{Sapling}}(d) = \perp \\ d, & \text{otherwise} \end{cases}$$

$$\text{DefaultDiversifier}(sk : \mathbb{B}^{[\ell_{sk}]}) := \text{first}(i : \mathbb{B}^{\mathbb{Y}} \mapsto \text{CheckDiversifier}(\text{truncate}_{(\ell_d/8)}(\text{PRF}_{sk}^{\text{expand}}([3, i]))) : \mathbb{J}^{(r)*} \cup \{\perp\}).$$

For a random *spending key*, $\text{DefaultDiversifier}$ returns $\perp$ with probability approximately 2^{-256} ; if this happens, discard the key and repeat with a different sk .

Notes:

- The protocol does not prevent using the *diversifier* d to produce “vanity” addresses that start with a meaningful string when encoded in *Bech32* (see § 5.6.3.1 ‘*Sapling Payment Addresses*’ on p. 119). Users and writers of software that generates addresses should be aware that this provides weaker privacy properties than a randomly chosen *diversifier*, since a vanity address can obviously be distinguished, and might leak more information than intended as to who created it.
- Similarly, address generators **MAY** encode information in the *diversifier* that can be recovered by the recipient of a payment to determine which *diversified payment address* was used. It is **RECOMMENDED**, instead of directly encoding information in the *diversifier*, to encode it in the *diversifier index* specified in [ZIP-32]. This ensures that the information is only accessible to a holder of the *diversifier key* dk .

Non-normative notes:

- Assume that $\text{PRF}^{\text{expand}}$ is a *PRF* with output range $\mathbb{B}^{[\ell_{\text{PRFexpand}}/8]}$, where $2^{\ell_{\text{PRFexpand}}}$ is large compared to $r_{\mathbb{J}}$.

Define $f : \mathbb{B}^{[\ell_{sk}]} \times \mathbb{B}^{\mathbb{Y}^{[N]}} \rightarrow \mathbb{F}_{r_{\mathbb{J}}}$ by $f_{sk}(t) := \text{ToScalar}^{\text{Sapling}}(\text{PRF}_{sk}^{\text{expand}}(t))$.

f is also a *PRF* since $\text{LEOS2IP}_{\ell_{\text{PRFexpand}}} : \mathbb{B}^{[\ell_{\text{PRFexpand}}/8]} \rightarrow \{0 \dots 2^{\ell_{\text{PRFexpand}}}-1\}$ is injective; the bias introduced by reduction modulo $r_{\mathbb{J}}$ is small because § 5.3 ‘*Constants*’ on p. 77 defines $\ell_{\text{PRFexpand}}$ as 512, while $r_{\mathbb{J}}$ has length 252 bits. It follows that the distribution of ask , i.e. $\text{PRF}_{sk}^{\text{expand}}([0]) : sk \xleftarrow{\mathbb{R}} \mathbb{B}^{[\ell_{sk}]}$, is computationally indistinguishable from $\text{SpendAuthSig}^{\text{Sapling}}.\text{GenPrivate}()$ defined in § 5.4.7.1 ‘*Spend Authorization Signature (Sapling and Orchard)*’ on p. 98.

- The distribution of nsk , i.e. $\text{ToScalar}^{\text{Sapling}}(\text{PRF}_{sk}^{\text{expand}}([1])) : sk \xleftarrow{\mathbb{R}} \mathbb{B}^{[\ell_{sk}]}$, is computationally indistinguishable from the uniform distribution on $\mathbb{F}_{r_{\mathbb{J}}}$. Since $nsk : \mathbb{F}_{r_{\mathbb{J}}} \mapsto \text{repr}_{\mathbb{J}}([nsk] \mathcal{H}^{\text{Sapling}} : \mathbb{J}_{\star}^{(r)})$ is bijective, the distribution of $\text{repr}_{\mathbb{J}}(nsk)$ will be computationally indistinguishable from uniform on $\mathbb{J}_{\star}^{(r)}$ (the keyspace of $\text{PRF}^{\text{nfSapling}}$).

4.2.3 Orchard Key Components

#orchardkeycomponents

Let $\ell_{\text{PRFexpand}}$, ℓ_{sk} , ℓ_{ovk} , ℓ_{d} , and ℓ_{dk} be as defined in § 5.3 ‘Constants’ on p. 77.

Let $\mathbb{P}$, $\text{repr}_{\mathbb{P}}$, $\ell_{\mathbb{P}}$, $q_{\mathbb{P}}$, and $r_{\mathbb{P}}$ be as defined in § 5.4.9.6 ‘Pallas and Vesta’ on p. 108.

Let $\text{Extract}_{\mathbb{P}}$ be as defined in § 5.4.9.7 ‘Coordinate Extractor for Pallas’ on p. 109.

Let $\text{GroupHash}_{\mathbb{P}}$ be as defined in § 5.4.9.8 ‘Group Hash into Pallas and Vesta’ on p. 110.

Let $\text{PRF}^{\text{expand}}$ and $\text{PRF}^{\text{ockOrchard}}$ be as defined in § 5.4.2 ‘Pseudo Random Functions’ on p. 89.

Let $\text{DeriveInternalFVK}^{\text{Orchard}}$ be as defined in [ZIP-32, Orchard internal key derivation].

Let $\text{PRP}^{\text{d}} : \mathbb{B}^{\ell_{\text{dk}}/8} \times \mathbb{B}^{\ell_{\text{d}}} \rightarrow \mathbb{B}^{\ell_{\text{d}}}$ be as defined in § 5.4.4 ‘Pseudo Random Permutations’ on p. 91.

Let $\text{KA}^{\text{Orchard}}$, instantiated in § 5.4.5.5 ‘Orchard Key Agreement’ on p. 93, be a key agreement scheme.

Let $\text{Commit}^{\text{ivk}}$, instantiated in § 5.4.8.4 ‘Sinsemilla commitments’ on p. 101, be a commitment scheme.

Let $\text{DiversifyHash}^{\text{Orchard}}$ be as defined in § 5.4.1.6 ‘DiversifyHash^{Sapling} and DiversifyHash^{Orchard} Hash Functions’ on p. 81.

Let $\text{SpendAuthSig}^{\text{Orchard}}$ instantiated in § 5.4.7.1 ‘Spend Authorization Signature (Sapling and Orchard)’ on p. 98 be a signature scheme with re-randomizable keys.

Let I2LEBSP , I2LEOSP , and LEOS2IP be as defined in § 5.1 ‘Integers, Bit Sequences, and Endianness’ on p. 76.

Define $\text{ToBase}^{\text{Orchard}}(x : \mathbb{B}^{\ell_{\text{PRFexpand}}/8}) := \text{LEOS2IP}_{\ell_{\text{PRFexpand}}}(x) \pmod{q_{\mathbb{P}}}$.

Define $\text{ToScalar}^{\text{Orchard}}(x : \mathbb{B}^{\ell_{\text{PRFexpand}}/8}) := \text{LEOS2IP}_{\ell_{\text{PRFexpand}}}(x) \pmod{r_{\mathbb{P}}}$.

Define $\text{DeriveDkAndOvk}^{\text{Orchard}}(\text{rivk} : \mathbb{F}_{r_{\mathbb{P}}}, \text{ak} : \{0 \dots q_{\mathbb{P}} - 1\}, \text{nk} : \mathbb{F}_{q_{\mathbb{P}}})$ as follows:

```

let  $K = \text{I2LEBSP}_{\ell_{\text{sk}}}(\text{rivk})$ 
let  $R = \text{PRF}_K^{\text{expand}}([0\text{x}82] \parallel \text{I2LEOSP}_{256}(\text{ak}) \parallel \text{I2LEOSP}_{256}(\text{nk}))$ 
let  $\text{dk} : \mathbb{B}^{\ell_{\text{dk}}/8}$  be the first  $\ell_{\text{dk}}/8$  bytes of  $R$  and let  $\text{ovk} : \mathbb{B}^{\ell_{\text{ovk}}/8}$  be the remaining  $\ell_{\text{ovk}}/8$  bytes of  $R$ .
return  $(\text{dk}, \text{ovk})$ 

```

A new Orchard *spending* key sk is generated by choosing a *bit* sequence uniformly at random from $\mathbb{B}^{\ell_{\text{sk}}}$.

From this *spending* key, the *Spend authorizing* key $\text{ask} : \mathbb{F}_{r_{\mathbb{P}}}^*$, the *Spend validating* key $\text{ak} : \{0 \dots q_{\mathbb{P}} - 1\}$, the *nullifier deriving* key $\text{nk} : \mathbb{F}_{q_{\mathbb{P}}}$, the $\text{Commit}^{\text{ivk}}$ randomness $\text{rivk} : \mathbb{F}_{r_{\mathbb{P}}}$, the *diversifier* key $\text{dk} : \mathbb{B}^{\ell_{\text{dk}}/8}$, the $\text{KA}^{\text{Orchard}}$ private key $\text{ivk} : \{1 \dots q_{\mathbb{P}} - 1\}$, the *outgoing viewing* key $\text{ovk} : \mathbb{B}^{\ell_{\text{ovk}}/8}$, and corresponding “internal” keys are derived as follows:

```

let mutable  $\text{ask} \leftarrow \text{ToScalar}^{\text{Orchard}}(\text{PRF}_{\text{sk}}^{\text{expand}}([6]))$ 
let  $\text{nk} = \text{ToBase}^{\text{Orchard}}(\text{PRF}_{\text{sk}}^{\text{expand}}([7]))$ 
let  $\text{rivk} = \text{ToScalar}^{\text{Orchard}}(\text{PRF}_{\text{sk}}^{\text{expand}}([8]))$ 
if  $\text{ask} = 0$ , discard this key and repeat with a new  $\text{sk}$ .
let  $\text{ak}^{\mathbb{P}} = \text{SpendAuthSig}^{\text{Orchard}}.\text{DerivePublic}(\text{ask})$ 
if the last bit (that is, the  $\tilde{y}$  bit) of  $\text{repr}_{\mathbb{P}}(\text{ak}^{\mathbb{P}})$  is 1:
    set  $\text{ask} \leftarrow -\text{ask}$ 
let  $\text{ak} = \text{Extract}_{\mathbb{P}}(\text{ak}^{\mathbb{P}})$ 
let  $\text{ivk} = \text{Commit}_{\text{rivk}}^{\text{ivk}}(\text{ak}, \text{nk})$ 
if  $\text{ivk} \in \{0, \perp\}$ , discard this key and repeat with a new  $\text{sk}$ .
let  $(\text{dk}, \text{ovk}) = \text{DeriveDkAndOvk}^{\text{Orchard}}(\text{rivk}, \text{ak}, \text{nk})$ 
let  $(\text{ak}_{\text{internal}}, \text{nk}_{\text{internal}}, \text{rivk}_{\text{internal}}) = \text{DeriveInternalFVK}^{\text{Orchard}}(\text{ak}, \text{nk}, \text{rivk})$ 

```

Each *Action description* is authorized by a signature, called the *spend authorization signature*.

Let $\ell_{\text{Merkle}}^{\text{Orchard}}$ be as defined in § 5.3 ‘*Constants*’ on p. 77.

Let $q_{\mathbb{P}}$ be as defined in § 5.4.9.6 ‘*Pallas and Vesta*’ on p. 108.

Let $\text{Extract}_{\mathbb{P}}$ be as defined in § 5.4.9.7 ‘*Coordinate Extractor for Pallas*’ on p. 109.

Let $\text{ValueCommit}^{\text{Orchard}}.\text{Output}$ be as defined in § 4.1.8 ‘*Commitment*’ on p. 32.

Let $\text{SpendAuthSig}^{\text{Orchard}}$ be as defined in § 4.15 ‘*Spend Authorization Signature (Sapling and Orchard)*’ on p. 59.

Let $\text{KA}^{\text{Orchard}}$ be as defined in § 4.1.5 ‘*Key Agreement*’ on p. 28.

Let Sym be as defined in § 4.1.4 ‘*Symmetric Encryption*’ on p. 28.

Let ZKAction be as defined in § 4.1.13 ‘*Zero-Knowledge Proving System*’ on p. 36.

Let vk be the Halo 2 verifying key identified by the Orchard circuit version, as specified in the consensus rules below.

An *Action description* comprises $(\text{cv}^{\text{net}}, \text{rt}^{\text{OrchardPool}}, \text{nf}, \text{rk}, \text{spendAuthSig}, \text{cm}_x, \text{epk}, \text{C}^{\text{enc}}, \text{C}^{\text{out}}, \text{enableSpends}, \text{enableOutputs}, \pi)$ where:

- $\text{cv}^{\text{net}} : \text{ValueCommit}^{\text{Orchard}}.\text{Output}$ is the *value commitment* to the value of the input *note* minus the value of the output *note*;
- $\text{rt}^{\text{OrchardPool}} : \{0 \dots q_{\mathbb{P}} - 1\}$ is an *anchor*, as defined in § 3.4 ‘*Transactions and Treestates*’ on p. 19, for the output *treestate* of a previous *block*;
- $\text{nf} : \{0 \dots q_{\mathbb{P}} - 1\}$ is the *nullifier* for the input *note*;
- $\text{rk} : \text{SpendAuthSig}^{\text{Orchard}}.\text{Public}$ is a randomized *validating key* that should be used to validate spendAuthSig ;
- $\text{spendAuthSig} : \text{SpendAuthSig}^{\text{Orchard}}.\text{Signature}$ is a *spend authorization signature*, validated as specified in § 4.15 ‘*Spend Authorization Signature (Sapling and Orchard)*’ on p. 59;
- $\text{cm}_x : \{0 \dots q_{\mathbb{P}} - 1\}$ is the result of applying $\text{Extract}_{\mathbb{P}}$ to the *note commitment* for the output *note*;
- $\text{epk} : \text{KA}^{\text{Orchard}}.\text{Public}$ is a *key agreement public key*, used to derive the key for encryption of the *transmitted note ciphertext* (§ 4.20 ‘*In-band secret distribution (Sapling and Orchard)*’ on p. 70);
- $\text{C}^{\text{enc}} : \text{Sym}.\text{C}$ is a ciphertext component for the encrypted output *note*;
- $\text{C}^{\text{out}} : \text{Sym}.\text{C}$ is a ciphertext component that allows the holder of the *outgoing cipher key* (which can be derived from a *full viewing key*) to recover the recipient *diversified transmission key* pk_d and the *ephemeral private key* esk , hence the entire *note plaintext*;
- $\text{enableSpends} : \mathbb{B}$ is a flag that is set in order to enable non-zero-valued spends in this *Action*;
- $\text{enableOutputs} : \mathbb{B}$ is a flag that is set in order to enable non-zero-valued outputs in this *Action*;
- $\text{enableCrossAddress} : \mathbb{B}$ is a flag that is set in order to enable the *expanded receiver* of the output *note* to be different from the *expanded receiver* of the input *note* in this *Action* [ZIP-2006];
- $\pi^{\text{OrchardPool}} : \text{ZKAction}.\text{Proof}$ is a *zk-SNARK proof* for the *Action statement* defined in § 4.18.4 on p. 66, with *primary input* $(\text{cv}, \text{rt}^{\text{OrchardPool}}, \text{nf}, \text{rk}, \text{cm}_x, \text{enableSpends}, \text{enableOutputs}, \text{disableCrossAddress})$ where $\text{disableCrossAddress} = 1 - \text{enableCrossAddress}$.

Note: The $\text{rt}^{\text{OrchardPool}}$, $\text{enableSpendsOrchard}$, and $\text{enableOutputsOrchard}$ components are the same for all *Action transfers* in a *transaction*, and are encoded once in the *transaction body* (§ 7.1 ‘*Transaction Encoding and Consensus*’ on p. 125), not the *ActionDescription* structure. $\pi^{\text{OrchardPool}}$ is aggregated with other *OrchardPoolAction* proofs and encoded in the proofsOrchard field.